

PROGRAMMING TASK 1 / 4 – SUBMISSION DEADLINE: MAY 25, 2026

Information

Exercises 1–5 in this sheet are based on Chapter 1 “Basics & tabular methods” of the lecture and cover the following topics:

- Multi-armed bandits
- Finite Markov decision processes
- Dynamic programming
- Monte-Carlo methods
- Temporal difference learning & Q-learning

To submit your solutions proceed as follows:

- Work on the programming exercises in groups of 3.
- Download the code for the programming tasks from the course Moodle page:
([Moodle - Programming Task 1](#)).
- In the file `programming_exercise01>README.txt` enter the full names and student ID numbers of all group members.
- Make the required changes to the files in the following exercises.
- After you have finished the exercise create a `zip`-file from your directory. Then only one person from your group submits the code via Moodle.

Exam admission requirements:

- Your solutions are evaluated on a pass / fail basis, and you will receive your result via Moodle.
- In the oral exam, there will also be questions related to your programming submissions.
- You need to pass 75% (i.e., 3 out of 4 programming tasks) to be allowed to take the exam.

Multi-armed bandits

Exercise 1.1 (Comparing ε -greedy and UCB on a 10-armed bandit)

We already discussed the ε -greedy algorithm for a positive constant $\varepsilon \in [0, 1]$, which we summarize in the following.

Algorithm 1 ε -greedy multi-armed bandit

```

Set  $Q_1(a) = 0$  for all  $a \in \mathcal{A}$  ;                               # Initialize action value estimates
Set  $N_1(a) = 0$  for all  $a \in \mathcal{A}$  ;                               # Initialize action counter
for  $t = 1, 2, \dots, n$  do
     $a_t \leftarrow \begin{cases} \arg \max_{a \in \mathcal{A}} Q_t(a) & \text{with probability } 1 - \varepsilon, \\ \text{random action} & \text{with probability } \varepsilon. \end{cases}$  ;           #  $\varepsilon$ -greedy action
    Sample  $r_t$  given action  $a_t$ ;                               # Sample from bandit
     $N_{t+1}(a) \leftarrow \begin{cases} N_t(a) + 1, & \text{if } a_t = a, \\ N_t(a), & \text{else.} \end{cases}$  ;           # Update action counter
     $Q_{t+1}(a) \leftarrow \begin{cases} Q_t(a) + \frac{1}{N_{t+1}(a)} [r_t - Q_t(a)], & \text{if } a_t = a, \\ Q_t(a), & \text{else.} \end{cases}$  ;           # Update action value
    estimate
end

```

In the programming exercise, we also consider the upper confidence bound algorithm with a positive constant $c > 0$.

Algorithm 2 Upper confidence bound (UCB) multi-armed bandit

```

Set  $Q_0(a) = 0$  for all  $a \in \mathcal{A}$  ;                               # Initialize action value estimate
Set  $N_0(a) = 0$  for all  $a \in \mathcal{A}$  ;                               # Initialize action counter
for  $t = 1, 2, \dots, n$  do
     $a_t \leftarrow \arg \max_{a \in \mathcal{A}} \left[ Q_{t-1}(a) + c \sqrt{\frac{\ln t}{N_{t-1}(a)}} \right]$  ; # UCB action,  $c > 0$  controls exploration
    Sample  $r_t$  given action  $a_t$  ;                               # Observe reward from bandit
     $N_t(a) \leftarrow \begin{cases} N_{t-1}(a) + 1, & \text{if } a_t = a, \\ N_{t-1}(a), & \text{else.} \end{cases}$  ;           # Update action counter
     $Q_t(a) \leftarrow \begin{cases} Q_{t-1}(a) + \frac{1}{N_t(a)} [r_t - Q_{t-1}(a)], & \text{if } a_t = a, \\ Q_{t-1}(a), & \text{else.} \end{cases}$  ;           # Update action value
    estimate
end

```

We use the convention $Q_{t-1}(a) + c \sqrt{\frac{\ln t}{N_{t-1}(a)}} = +\infty$ if $N_{t-1}(a) = 0$ to deal with division by zero, i.e., an action that has not been taken so far is a maximizing action.

In this exercise, you will implement and compare multi-armed bandit strategies: ε -greedy (with fixed and time-varying exploration) and upper confidence bound (UCB). We consider a 10-armed bandit with each arm $a \in \{1, \dots, 10\}$ having a reward distribution $p(r_t | a_t = a) \sim \mathcal{N}(\mu_a, 1)$, where the means μ_a are sampled once from $\mathcal{N}(0, 1)$ at the start of the experiment.

1.1.1 (Environment setup)

Open the file `programming_exercise_01>multiarmed_bandits>functions_01_1.py` and edit the class `MultiarmedBandit`.

- a) Edit the method `__init__` to initialize a multi-armed bandit given a list of means $[\mu_1, \dots, \mu_K]$.
- b) Edit the method `sample` that returns a reward $p(r_t | a_t = a) \sim \mathcal{N}(\mu_a, 1)$ for a chosen arm $a \in \{1, \dots, K\}$.

1.1.2 (ϵ -greedy algorithm)

Open the file `programming_exercise_01>multiarmed_bandits>functions_01_1.py` and edit the function `epsilon_greedy()` that implements the ϵ -greedy multi-armed bandit algorithm as described in Algorithm 1. In your implementation include the option for time varying exploration rates $\epsilon_t = \frac{\delta}{t}$ for $\delta \in [0, 1]$.

1.1.3 (UCB algorithm)

In `programming_exercise_01>multiarmed_bandits>functions_01_1.py` edit the function `ucb()` that implements the upper confidence bound multi-armed bandit algorithm as described in Algorithm 2.

1.1.4 (Comparison)

Compare your algorithms using `programming_exercise_01>multiarmed_bandits>exercise_01_1.py`. Define a 10-armed bandit with each arm $a \in \{1, \dots, 10\}$ having a reward distribution $p(r_t | a_t = a) \sim \mathcal{N}(\mu_a, 1)$, where the means μ_a are sampled once from $\mathcal{N}(0, 1)$ at the start of the experiment. Compare the following policies:

- i) ϵ -greedy with a fixed exploration rate of $\epsilon = 0.1$.
- ii) ϵ -greedy with a time varying exploration rate of $\epsilon_t = \frac{\delta}{t}$ with $\delta = 1$.
- iii) UCB with parameter $c = 2$

Evaluate all algorithms using $n = 10,000$ actions.

- a) Plot the average reward over time for all policies in one graph.
- b) Plot the percentage of optimal actions over time for all policies in one graph.
- c) Plot the regret over time for all policies in one graph.

Markov decision processes - Grid world

In the following exercises, you implement a grid world environment to investigate the tabular reinforcement learning methods discussed in the lecture. The task involves a robot agent moving on a finite grid. Its objective is to reach a battery in order to recharge its energy.

The robot operates under significant constraints. Although it has a friendly smile, it has no perception of its surroundings and can only interact with the environment through its actions. It only discovers the battery's location upon reaching it. Additionally, the grid contains obstacles that the robot cannot traverse, making the task more challenging.

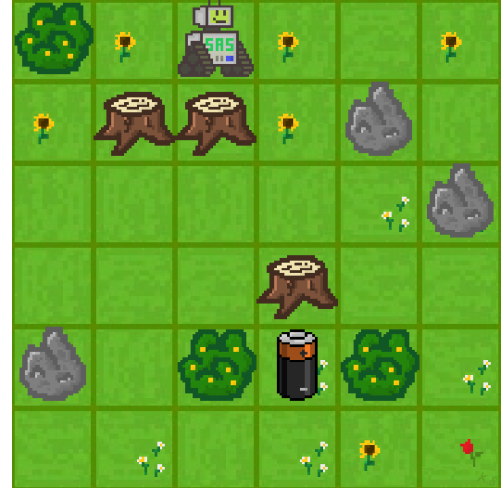


Figure 1: Example grid world with robot agent, battery and various obstacles

To help the robot reach the battery, you must put your reinforcement learning knowledge into action!

Exercise 1.2 (Creating the Grid World Environment)

In this exercise, you will implement a grid world environment and explore its behavior. The environment is a square grid of size $N \times N$ with an agent (robot), a goal (battery), and obstacles. The agent can move *Up*, *Down*, *Left*, *Right*, and transitions are deterministic: For a given action the agent moves to the adjacent field in the desired direction if there is no obstacle. If in this direction there is an obstacle or the boundary of the grid world it remains at its position. To implement the environment open the file `programming_exercise_01>grid_world>gridworld.py` and change the methods of the class `GridWorld` according to the following subexercises.

1.2.1 (Environment setup)

a) Investigate the structure of the `GridWorld` class. The class `GridWorld` gets initialized with the following parameters.

- **size:** grid size N
- **battery_pos:** (x, y) tuple, the position of the battery
- **obstacles:** list of (x, y) tuples, the positions of the obstacles

b) The class `GridWorld` has the following methods which will be edited in the following exercises.

- **states():** Returns a set of all valid states in the gridworld (i.e., all positions that are not obstacles).
- **successors(state):** Returns a set of valid successor states that can be reached from the given state by taking any of the possible actions, i.e., all states s' with $\sum_{a \in \mathcal{A}} p(s'|s, a) > 0$.
- **p(successor, state, action):** Returns the probability of transitioning from state to successor using the given action.
- **step(state, action):** Given a state and an action, returns the successor for the transition under action.
- **reward(successor, state, action):** Given a state and an action, returns the successor for the transition under action.
- **render(state):** Prints the grid world to the console, with the agent's current position marked as 'X', obstacles marked as '#', and the battery marked as 'B'.

1.2.2 (State representation)

Edit the method `states()`. The method should return a list of all possible states, i.e., of all fields in the $N \times N$ grid which are not obstacles.

1.2.3 (Successors)

Edit the method `successors(state)`. For a given state this method should return a list with all possible successor states. The state with the battery position is a terminal state. Take obstacles and boundaries of the grid into account.

1.2.4 (Transition probabilities)

Edit the method `p(successor, state, action)`. For a given state, action and successor state this method should return the transition probability. The agent follows the action deterministically. The state with the battery position is a terminal state. Take obstacles and boundaries of the grid into account.

1.2.5 (Transitions)

Edit the method `step(state, action)`. Given a state and an action sample the successor state according to the probabilities given by the function `p(successor, state, action)`.

1.2.6 (Reward)

Edit the method `reward(successor, state, action)`. For a given state, an action and a successor state this function should return the current reward. We impose the following model assumptions for the grid world.

- Whenever the agent transitions from a state which is not the battery position to the battery position, return reward $r = 1$.
- In all other cases, return reward $r = 0$.

1.2.7 (Visualization)

The last method of the class is `render(state)`. This method renders the environment and the current state by printing it to the terminal. You do not have to make any changes to this method.

1.2.8 (Testing)

Open and run the file `programming_exercise_01>grid_world>exercise_01_2.py`. This file instantiates a grid world environment and renders it for a fixed state. Feel free to use this file interactively for testing of your implementation.

Dynamic programming

Exercise 1.3 (Value Estimation)

In this exercise, we consider value estimation methods for the `GridWorld` environment implemented in the preceding exercise. We investigate the Bellman expectation equality and iterative policy evaluation. After implementing the necessary functions in the beginning of this exercise, we conduct experiments on the example grid world instance shown in Figure 2. The functions for this exercise are contained in the file `programming_exercise_01>grid_world>functions_01_3.py`, while the code for experiments is given in `programming_exercise_01>grid_world>exercise_01_3.py`



Figure 2: Example grid world of size 4×4 with robot agent, battery, and obstacles

1.3.1 (Bellman expectation equality)

Edit the function `solve_Bellman_expectation(env, pi, gamma)` which computes for a given policy π and a discount factor γ the state values V^π by solving the Bellman expectation equality

$$V^\pi(s_t) = \mathbb{E}_\pi[r_t | s_t] + \gamma \sum_{s_{t+1} \in \mathcal{S}} V^\pi(s_{t+1}) \cdot p(s_{t+1} | s_t, \pi(s_t)).$$

This equation can be solved in the following way. First compute the mean rewards $\hat{r}_\mathcal{S} = [\hat{r}_1, \dots, \hat{r}_n]^\top$ with $\hat{r}_i = \mathbb{E}_\pi[r | s_i]$ and define $V_\mathcal{S} = [V(s_1), \dots, V(s_n)]^\top$ and the transition matrix $P \in \mathbb{R}^{n \times n}$ with $P_{ij} = p(s_j | s_i, \pi(s_i))$. Then, the state values can be obtained by solving the system of equations

$$(I - \gamma P)V_\mathcal{S} = \hat{r}_\mathcal{S}.$$

1.3.2 (Iterative policy evaluation)

Edit the function `iterative_policy_evaluation(env, pi, gamma, max_iters)` to implement the iterative policy evaluation algorithm ([Lecture 03 - Slide 16](#)). This method takes

- a `GridWorld` object `env`,
- a policy `pi` mapping states to probability distributions over actions,
- a discount factor `gamma`,
- the number of iterations `max_iter` after which the algorithm stops,
- a tolerance `tol` for the stopping condition,
- a dictionary `V_true` to monitor the distance to the true value function,

and computes the value function, using the iterative policy evaluation algorithm. Only make changes in the inner loop where it is written `# MAKE CHANGES HERE`.

1.3.3 (Plot value functions)

In `programming_exercise_01>grid_world>functions_01_3.py` you find the function `plot(env, V)`. You do not have to make any changes here. Use this function to plot a value function as a heat map. An example for random values assigned to the states of the grid world shown in in Figure 2 is visualized in Figure 3.

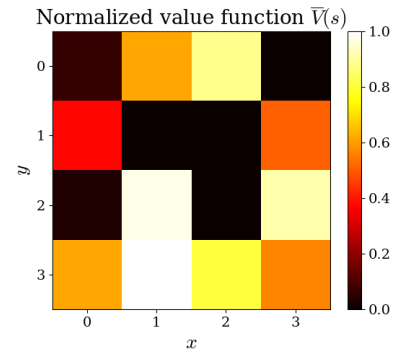


Figure 3: Heat map for a random function $V^{\text{random}}(s)$

1.3.4 (Experiments for an example environment) In this exercise, we investigate the functions defined previously. For all experiments use the environment shown in Figure 2. All experiments should be contained in the file `programming_exercise_01>grid_world>exercise_01_3.py`

- a) Create an environment with robot, battery and obstacles as shown in Figure 2. Test the function `solve_Bellman_expectation()` and `plot()` to compute and visualize the value functions for the following settings.
 - i) Define a policy π that chooses the actions right and down with 50/50 probability and set discount factor $\gamma = 0.9$.
 - ii) Define a policy π that chooses the actions right or down with 50/50 probability and set discount factor $\gamma = 0.1$.
 - iii) Define a policy π that chooses the action down with probability 1 and set discount factor $\gamma = 0.9$.
 - iv) Try to guess an optimal policy π^* and set discount factor $\gamma = 0.9$.
- b) Define the policy π that chooses the actions right or down with 50/50 probability. Compare the convergence of $\|V_n^\pi - V^\pi\|_\infty$ for $\gamma = 0.9$ and $\gamma = 0.1$ by computing the true value function V^π for both values of γ using `solve_Bellman_expectation()`. Visualize the convergence of $\|V_n^\pi - V^\pi\|_\infty$ by plotting both curves in the same plot. You find a suggestion for the plots in the file `programming_exercise_01>grid_world>exercise_01_3.py`

Monte-Carlo methods

Exercise 1.4 (Value Estimation)

In this exercise, we implement two versions of Monte-Carlo value estimation. All functions for this exercise are contained in `programming_exercise_01>grid_world>functions_01_4.py`. The code for the experiments is included in `programming_exercise_01>grid_world>exercise_01_4.py`

1.4.1 (Every-visit Monte-Carlo method)

In the file `programming_exercise_01>grid_world>functions_01_4.py`, implement the every-visit Monte-Carlo method for function value estimation by editing the function `ev_monte_carlo(env, pi, gamma, s0, n_samples, T, V_true)` with input parameters

- `env` an object of the `GridWorld` class,
- `pi` a given policy,
- `gamma` the discount factor,
- `s0` the initial state,
- `n_samples` the number of episodes,
- `T` the length of episodes,
- `V_true` the true value function for diagnostics.

The function should return

- `V` the value estimates,
- `dist` the distances of `V` to the true value function `V_true` per iteration if these values are provided.

Change the code in the loop where it is written `# MAKE CHANGES HERE` to implement the every-visit Monte-Carlo method.

1.4.2 (First-visit Monte-Carlo method)

This subexercise is analogous to previous one.

In the file `programming_exercise_01>grid_world>functions_01_4.py`, implement the first-visit Monte-Carlo method for function value estimation by editing the function `fv_monte_carlo(env, pi, gamma, s0, n_samples, T, V_true)` with input parameters

- `env` an object of the `GridWorld` class,
- `pi` a given policy,
- `gamma` the discount factor,
- `s0` the initial state,
- `n_samples` the number of episodes,
- `T` the length of episodes,
- `V_true` the true value function for diagnostics.

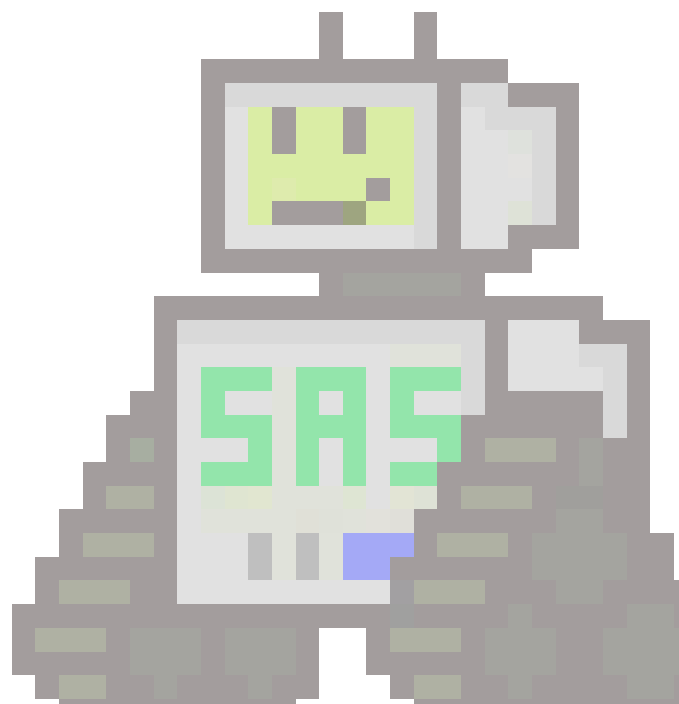
The function should return

- `V` the value estimates,
- `dist` the distances of `V` to the true value function `V_true` per iteration if these values are provided.

Change the code in the loop where it is written `# MAKE CHANGES HERE` to implement the first-visit Monte-Carlo method.

1.4.3 (Experiment for comparison)

Compare your implementation of the every-visit and first-visit Monte-Carlo method. The code for the comparison is contained in `programming_exercise_01>grid_world>exercise_01_4.py`. As an environment choose the grid world instance shown in Figure 2. Fix the discount factor $\gamma = 0.9$ and the policy that chooses the action to go down or right with 50/50 probability. Starting from the initial state $s_0 = (0,0)$, perform an experiment with 20.000 samples and an episode length of $T = 128$. To compare the obtained value estimates to the true values, we first compute the true value function using the function `solve_Bellman_expectation` from Exercise 1.3.4. As this is a stochastic method we perform this experiment 20 times and compute the empirical mean and standard deviation for comparison and plot the results in one figure. Once you adapted the code in `programming_exercise_01>grid_world>functions_01_4.py` it is enough to run the code in `programming_exercise_01>grid_world>exercise_01_4.py` without further changes.



Temporal difference learning & Q-learning

Exercise 1.5

Implement temporal difference learning for value estimation and the SARSA algorithm and Q-learning algorithm for control. The file `programming_exercise_01>grid_world>functions_01_5.py` contains the functions, while the code for the experiments is given in:

- `programming_exercise_01>grid_world>exercise_01_5a.py`
- `programming_exercise_01>grid_world>exercise_01_5b.py`

1.5.1 (TD-based prediction)

In `programming_exercise_01>grid_world>functions_01_5.py`, implement the TD-based prediction algorithm by adapting the function `td_prediction(env, pi, gamma, alpha, s0, n_samples, T, V_true)`. The function has the input parameters

- `env` an object of the `GridWorld` class,
- `pi` a given policy,
- `gamma` the discount factor,
- `alpha` the step size
- `s0` the initial state,
- `n_samples` the number of episodes,
- `T` the length of episodes,
- `V_true` the true value function for diagnostics,

and returns

- `V` the value estimates,
- `dist` the distances of `V` to the true value function `V_true` per iteration if these values are provided.

Change the code in the loop where it is written `# MAKE CHANGES HERE` to implement the TD-based prediction algorithm.

1.5.2 (Experiment)

Evaluate the TD-based prediction algorithm similar to Exercise 1.4.3. for the Monte Carlo methods. The file `programming_exercise_01>grid_world>exercise_01_5a.py` contains the code for the experiments. As an environment choose the grid world instance shown in Figure 2. Fix the discount factor $\gamma = 0.9$ and the policy that chooses the action to go down or right with 50/50 probability. Starting from the initial state `s_0 = (0,0)`, perform an experiment with 20.000 samples and an episode length of $T = 128$. To compare the obtained value estimates to the true values, we first compute the true value function using the function `solve_Bellman_expectation` from Exercise 1.3.4. As this is a stochastic method we perform this experiment 20 times and compute the empirical mean and standard deviation. Compare the results against the ones obtained for `fv_monte_carlo()` and `ev_monte_carlo()`. Once you adapted the code in `programming_exercise_01>grid_world>functions_01_5.py` it is enough to run the code in `programming_exercise_01>grid_world>exercise_01_5a.py` without further changes.

1.5.3 (On-policy TD control: SARSA)

In `programming_exercise_01>grid_world>functions_01_5.py`, implement On-policy TD control: SARSA by adapting the function `SARSA(env, gamma, alpha, eps, s0, n_samples, T)`. The function has the input parameters

- `env` an object of the `GridWorld` class,
- `gamma` the discount factor,
- `alpha` the step size,
- `eps` the exploration rate,
- `s0` the initial state,
- `n_samples` the number of episodes,
- `T` the length of episodes.

and returns

- `Q` the value estimates,
- `pi` the obtained policy.

Change the code in the loop where it is written `# MAKE CHANGES HERE` to implement On-policy TD control: SARSA.

1.5.4 (Off-policy TD control: Q-learning)

In `programming_exercise_01>grid_world>functions_01_5.py`, implement Off-policy TD control: Q-learning by adapting the function `Q(env, gamma, alpha, eps, s0, n_samples, T)`. The function has the input parameters

- `env` an object of the `GridWorld` class,
- `gamma` the discount factor,
- `alpha` the step size,
- `eps` the exploration rate,
- `s0` the initial state,
- `n_samples` the number of episodes,
- `T` the length of episodes.

and returns

- `Q` the value estimates,
- `pi` the obtained policy.

Change the code in the loop where it is written `# MAKE CHANGES HERE` to implement Off-policy TD control: Q-learning.

1.5.5 (Experiment for SARSA and Q-learning)

Compare your implementation of SARSA and Q-learning. The file `programming_exercise_01>grid_world>exercise_01_5b.py` contains the code for comparison. As an environment choose the grid world instance shown in Figure 1. Fix the discount factor `gamma = 0.9`, step size `alpha = 0.1`, exploration rate `eps = 0.1` and the initial policy that chooses among all actions with 25/25/25/25 probability. Starting from the initial state `s_0 = (0,2)`, perform an experiment with 200 samples and an episode length of $T = 100$. To compare the obtained value estimates, first compute the policies using the methods `SARSA()` and `Q_learning()` from Exercise 1.5.4. Afterwards, compute the value functions using `solve_Bellman_expectation()` from Exercise 1.3.1. and visualize the results using `plot_V()`.

After updating `programming_exercise_01>grid_world>functions_01_5.py` it is enough to run the code in `programming_exercise_01>grid_world>exercise_01_5b.py` without further changes.

